LendFlow

AI Loan Underwriting Pipeline

Interview Defense Pack

Stack	LangGraph · FastAPI · ChromaDB · BM25 · RAGAS · Docker
Nodes	7 — intake · policy_check · rag_lookup · foir_engine · risk_scorer · decision · audit
Key Pattern	Deterministic-first · Hybrid RAG · LLM for synthesis only
Evaluation	RAGAS faithfulness 0.91 · Routing accuracy 95% · PII egress 0
Author	Sidharth Kriplani — github.com/SidharthKriplani/lendflow

25+ Q&A pairs · Architecture walkthrough · Design decisions · Production considerations · Portfolio comparison

1. Architecture Walkthrough — 7 Nodes

LendFlow implements a deterministic-first LangGraph pipeline. The graph runs 7 nodes in sequence, with early-exit at Node 2 if hard RBI/NBFC rules fail. Only Nodes 3 and 6 involve non-deterministic components (retrieval and LLM synthesis respectively). All numeric underwriting decisions are produced by deterministic nodes before the LLM is ever called.

Node	Name	Type	Key Output
1	intake	Deterministic	PII-redacted state, alert_id, source tag
2	policy_check	Deterministic (no LLM)	PASS or HARD_REJECT with rule code
3	rag_lookup	Hybrid retrieval	8 ranked policy chunks with citations
4	foir_engine	Deterministic (no LLM)	FOIR value, threshold band
5	risk_scorer	Rule-based + bureau	Risk band A/B/C/D, score components
6	decision	LLM synthesis	APPROVE/REFER/REJECT + product + EMI
7	audit	Deterministic	audit_id, JSONL record

Q: Walk me through the full pipeline for a vehicle loan application.

The application arrives as raw text through the FastAPI endpoint. Node 1 (intake) runs Presidio and custom regex recognizers to mask name, Aadhaar, PAN, and phone number in-place, assigns a unique alert_id, and tags the source (branch/API/mobile app). The redacted state is passed to Node 2 (policy_check), which runs six deterministic RBI/NBFC rule checks — LTV cap, age limits, income floor, blacklisted pincode, vehicle age, loan tenor — and short-circuits to HARD_REJECT immediately if any fail. If all pass, Node 3 (rag_lookup) runs hybrid BM25+dense retrieval over the three-corpus policy index, reranks with a cross-encoder, and returns 8 ranked policy chunks. Node 4 (foir_engine) recomputes FOIR from raw figures and maps it to a threshold band. Node 5 (risk_scorer) builds a risk band from bureau score, LTV ratio, employment type, and vintage. Node 6 (decision) receives all prior outputs plus the policy chunks and calls the LLM to synthesize a final decision, product recommendation, and EMI calculation. Node 7 (audit) writes a JSONL record with audit_id. Total latency: approximately 1.2 seconds.

Q: Why LangGraph instead of a simple sequential function call chain?

LangGraph provides three things a function chain cannot: (1) typed state management via LoanState TypedDict — every node reads and writes a shared, validated state

object; (2) conditional edges — the HARD_REJECT path from policy_check skips Nodes 3-6 and jumps directly to audit, which is cleaner and more efficient than exception handling in a linear chain; (3) built-in interrupt_before support for human-in-the-loop review on REFER decisions without custom threading logic.

Q: What is in the LoanState TypedDict?

LoanState carries: raw_text (redacted), alert_id, source, policy_check_result (with rule codes), rag_chunks (list of PolicyChunk with text and citation), foir (float), foir_band (str), risk_band (str), bureau_score (int), decision (str), product (str), emi (float), policy_citations (list), confidence (float), token_count (int), latency_ms (int), and audit_id. PII is never stored in state — only the redacted text.

2. Why Deterministic-First?

Q: Why do you recompute FOIR instead of using the value from the application?

Because the application-provided FOIR is untrustworthy. Loan applications frequently contain incorrect, manipulated, or incomplete EMI declarations. An applicant may omit existing credit card EMIs or personal loan obligations. A branch officer may enter estimated figures. If we trust the input FOIR, we are making a lending decision on unverified data — which violates the spirit of RBI's responsible lending guidelines and exposes the NBFC to regulatory risk. Node 4 recomputes FOIR from the gross income figure (extracted and independently validated), declared EMI obligations (cross-referenced where possible), and the proposed EMI calculated from the requested loan amount and tenor. This is always the authoritative value.

FOIR Formula and Thresholds

```
FOIR = (Sum of all fixed monthly obligations + proposed EMI) / Net monthly income
```

FOIR Range	Decision Band	RBI Alignment
≤ 0.50	APPROVE tier	Conservative — RBI recommends ≤ 0.50 for vehicle loans
0.51 – 0.65	REFER — manual review	Borderline — officer judgment required
> 0.65	REJECT	Exceeds safe lending threshold

Q: Why does policy_check use no LLM at all?

Hard regulatory rules are binary and deterministic — an LTV ratio either exceeds the RBI cap or it does not. There is no ambiguity to resolve, no synthesis required, no nuance the LLM could add. Using an LLM for binary threshold checks introduces latency, token cost, hallucination risk (the model could incorrectly classify a rule as passing), and unpredictability. Deterministic Python conditionals are 100% reliable, zero-cost, and trivially auditable. The auditor reviewing a HARD_REJECT can read the exact Python condition that triggered it — no black box.

Q: What are the six hard rules in policy_check?

1. LTV cap: loan amount / vehicle value must not exceed the RBI-mandated LTV for the vehicle category. 2. Applicant age: borrower must be between 21 and 65 at loan origination. 3. Income floor: net monthly income must meet the NBFC's minimum income threshold (typically ■25,000 for vehicle loans). 4. Pincode blacklist: applicant's residence or office pincode must not appear on the NBFC's blacklisted geographies list. 5. Vehicle age: for used vehicles, the vehicle's manufacturing year must fall within

the permissible window. 6. Loan tenor: requested tenor must not exceed the NBFC's maximum permissible tenor for the vehicle class.

3. Hybrid RAG over Credit Policy

Q: Why hybrid retrieval (BM25 + dense) instead of just vector search?

Credit policy documents are structurally different from general text. They contain precise regulatory phrases — 'FOIR shall not exceed 0.55 for salaried borrowers', 'LTV ratio for new vehicles shall not exceed 85%' — that dense embedding models may rank poorly because they optimize for semantic similarity, not exact phrase match. BM25 is a keyword frequency model that excels at exact-match retrieval of these regulatory clauses. Dense retrieval (ChromaDB + sentence-transformers) excels at semantic generalization — finding relevant policy chunks even when the query uses different terminology than the document. Hybrid retrieval merges both candidate sets and runs a cross-encoder reranker to produce a final ranked list that is better than either method alone.

Q: Describe the three corpora and why each is needed.

1. RBI Master Directions: the primary regulatory corpus — defines FOIR limits, LTV caps, income floor guidance, and KYC requirements for all NBFCs. Without this, the system cannot cite authoritative regulatory basis for any decision. 2. NBFC Sector-Specific Circulars: vehicle lending specific — covers used vehicle policies, sectoral LTV adjustments, employment type weightings, and product eligibility. 3. NHB Circulars: overlapping coverage for vehicle loans financed through housing finance companies — captures edge cases in cross-sector lending products. Each corpus covers regulatory territory not fully addressed by the others.

Q: How does the cross-encoder reranker work and why is it better than just merging BM25 and dense scores?

The cross-encoder (cross-encoder/ms-marco-MiniLM-L-6-v2) takes a (query, candidate_passage) pair as input and produces a single relevance score — it sees both query and passage together, unlike bi-encoder models that encode them independently. This allows it to model fine-grained query-passage interactions that independent embedding scores miss. The cost is higher latency (O(n) inference calls for n candidates), so the cross-encoder only re-scores a small merged candidate set (typically 20-30 candidates from BM25 + dense), not the full corpus. The reranker's output is the final ranked list passed to the decision node.

Q: How do you evaluate RAG quality?

Using RAGAS metrics: (1) Faithfulness — does the LLM decision reference only claims supported by the retrieved chunks? Scored 0.91. (2) Context recall — do the retrieved chunks contain all information needed for the correct decision? (3) Answer relevancy — is the synthesized rationale relevant to the query? These metrics are computed by a RAGAS evaluator LLM over a test set of 20 applications with

ground-truth policy citations. The pipeline also supports manual citation verification — every policy citation in the output maps to a specific retrieved chunk.

4. LangGraph State Design

Q: What is the LoanState TypedDict and why TypedDict over a Pydantic model?

LoanState is a TypedDict that defines the shared mutable state passed between all 7 nodes. TypedDict is LangGraph's native state container — it supports partial updates (each node only writes its output fields, not the entire state), and LangGraph's checkpointing mechanism serializes TypedDict state natively. A Pydantic model would require custom serialization hooks and doesn't natively support LangGraph's reducer pattern for merging partial state updates. That said, all node outputs are validated against Pydantic schemas before being written to state.

Q: How does the HARD_REJECT conditional edge work?

The graph uses a conditional edge after Node 2 (policy_check). The routing function reads state['policy_check_result']['passed'] — if False, it returns 'audit' as the next node, bypassing Nodes 3-6. If True, it returns 'rag_lookup'. This means a failed application runs only 2 nodes (intake + policy_check) instead of all 7, saving approximately 1 second of latency and all RAG and LLM costs.

Q: How does human-in-the-loop review work for REFER decisions?

LangGraph's interrupt_before mechanism pauses execution before Node 6 (decision) when the FOIR band is REFER (0.51-0.65). The application state is persisted in the LangGraph checkpoint store. The FastAPI endpoint returns a 202 status with a thread_id. The human review API (POST /human-review/override) accepts the thread_id and an officer decision (APPROVE/REJECT with reason), resumes the graph from the checkpoint, and Node 6 incorporates the officer decision into the final output. The audit log records both the system recommendation and the officer override.

5. RAGAS Evaluation Framework

Q: What RAGAS metrics do you track and what do they measure?

Three core RAGAS metrics: (1) Faithfulness measures whether every claim in the LLM's decision rationale is supported by the retrieved policy chunks — a faithfulness of 1.0 means zero hallucinated regulatory claims. Our score is 0.91. (2) Context Recall measures whether the retrieved chunks contain all the relevant policy information needed for a correct decision — computed against ground-truth policy citations. (3) Answer Relevancy measures whether the synthesized decision rationale is responsive to the loan application query — low scores indicate the LLM is off-topic or verbose without substance.

Q: How do you generate the evaluation test set?

20 synthetic NBFC loan applications are generated programmatically across 4 document types (bank statements, salary slips, KYC documents, vehicle inspection reports). Each application has a ground-truth JSON file specifying the correct routing decision, the FOIR value, the relevant policy citations, and whether each policy rule should pass or fail. The eval harness (scripts/run_eval.py) runs each application through the pipeline and computes routing accuracy, FOIR computation accuracy, PII recall, PII egress, and RAGAS metrics.

Q: Your routing accuracy is 95% (19/20). What was the failure case?

The one failure was a borderline FOIR case (0.497) that the system classified as APPROVE but the ground truth was REFER — the ground truth reflected a manual underwriter's judgment that the applicant's income documentation quality warranted additional review despite the FOIR being technically within the approval threshold. This is a known limitation: FOIR thresholds are hard boundaries, but experienced underwriters apply qualitative judgment in near-threshold cases. Production deployment would add a near-threshold buffer zone (e.g., 0.48-0.50 → REFER) configurable per NBFC policy.

6. Indian NBFC Regulatory Context

Q: What is the regulatory basis for the FOIR threshold in Indian vehicle lending?

The Reserve Bank of India's Master Direction on Non-Banking Financial Companies — Systemically Important Non-Deposit taking Company and Deposit taking Company (Reserve Bank) Directions, 2016 provides the framework. The FOIR concept (Fixed Obligation to Income Ratio) is a key affordability metric RBI expects NBFCs to apply. Industry practice for vehicle loans is ≤ 0.50 for APPROVE — this is more conservative than the RBI's own guidance of ≤ 0.55, reflecting typical NBFC risk appetite for vehicle asset class. The threshold is configurable in config.py to allow per-NBFC calibration.

Q: What is the PII handling approach and why does it matter for Indian financial regulation?

India's Digital Personal Data Protection Act 2023 (DPDPA) and the existing IT Act mandate data minimization — only necessary personal data should be processed, and it must be protected. LendFlow redacts PII (name, Aadhaar number, PAN, phone) before the text enters the state that is passed to the LLM and written to audit logs. The PII mapping is stored only in memory (or an encrypted KV store in production) and never in the audit log. The audit log contains only a pii_map_id pointer. This means the audit log is safe to write to any storage without triggering DPDPA data residency obligations for PII.

Q: How would you handle a regulatory audit of a loan decision?

Every loan decision produces a JSONL audit record (Node 7) containing: the alert_id, the policy_check_result with individual rule outcomes and codes, the FOIR recomputed value, the risk band, the decision, the product, and the exact policy citations from the RAG chunks. The audit record is immutable — Node 7 writes it once, it is never modified. A regulatory auditor can trace any APPROVE/REJECT to the exact RBI/NBFC rule or policy clause that determined the outcome. This is the core value of deterministic-first design: human-readable audit trails with no black box.

7. Production Considerations

Q: How would you integrate with a real credit bureau (CIBIL/Experian)?

Node 5 (risk_scorer) is designed for bureau integration. In the current implementation it uses a mock bureau score. In production: (1) The applicant's PAN (stored as pii_map_id pointer, not plaintext) would be used to call the bureau API inside a secure PII-handling context. (2) The bureau response (score, default history, tradeline summary) would be parsed and injected into state before risk_scorer runs. (3) Bureau API latency (typically 200-500ms) would be included in the total latency budget. The pipeline's node design isolates bureau integration to a single node, making it swappable without touching the rest of the graph.

Q: What is the latency SLA and how do you meet it?

Current pipeline latency is approximately 1.2 seconds for an APPROVE path (all 7 nodes). HARD_REJECT paths (2 nodes) are under 50ms. The RAG retrieval (Node 3) is the largest contributor at approximately 300ms for BM25 + dense retrieval + reranking. The LLM synthesis (Node 6) contributes approximately 600-800ms depending on model and token count. For production SLAs of under 2 seconds: (1) ChromaDB can be replaced with a served vector database (Weaviate/Qdrant) for faster retrieval. (2) The cross-encoder can be served as a separate microservice. (3) Node 4 (foir_engine) and Node 5 (risk_scorer) can run in parallel with Node 3 since they don't depend on RAG output.

Q: How would you scale this to 10,000 applications per day?

At 10,000 applications/day the system needs to process approximately 7 applications per minute, which the current single-instance FastAPI handles comfortably. For higher throughput: (1) The LangGraph pipeline is stateless per application — horizontal scaling via multiple FastAPI replicas behind a load balancer is trivial. (2) ChromaDB can be replaced with a distributed vector database. (3) The LLM call (Node 6) is the primary bottleneck at scale — a batching queue with async processing and LLM provider autoscaling handles burst load. (4) HARD_REJECT applications (typically 15-20% of volume) never reach the LLM, reducing LLM cost proportionally.

Q: How would you detect model drift over time?

Two monitoring dimensions: (1) Decision distribution drift — track the APPROVE/REFER/REJECT ratio daily. A sudden shift (e.g., APPROVE rate drops from 60% to 40%) indicates either population shift or a retrieval quality problem. (2) RAGAS faithfulness drift — run a weekly eval on a held-out test set. If faithfulness drops below 0.85, the RAG index may be stale (new RBI circulars not indexed) or the LLM synthesis quality has degraded. The deterministic nodes (policy_check, foir_engine) don't drift — they are hardcoded rules. Only the RAG retrieval and LLM

synthesis components require drift monitoring.

8. Hard Interview Questions

Q: What if FOIR is exactly 0.50? Which band does it fall into?

FOIR = 0.50 falls into the APPROVE tier. The threshold is defined as: ≤ 0.50 → APPROVE, 0.51-0.65 → REFER, >0.65 → REJECT. The boundary is inclusive on the APPROVE side — 0.50 is exactly at the regulatory guideline and qualifies for approval. This is a deliberate design choice: the threshold should match the regulatory guidance precisely, not introduce an additional safety margin that isn't in the regulation (that decision belongs to the NBFC credit committee, not the pipeline). The foir_engine uses >= and <= comparisons, not strict inequalities, and the thresholds are configurable in config.py.

Q: What happens if the cross-encoder reranker crashes? Does the pipeline fail?

Node 3 (rag_lookup) has a fallback path: if the cross-encoder reranker raises an exception, the node falls back to returning the top-k candidates from the BM25+dense merge by raw score (without reranking). The decision node receives fewer-quality chunks but the pipeline completes. The fallback is logged and flagged in state as 'reranker_fallback: true'. In production, the reranker would be served as a separate microservice with health checks and circuit breaker logic, making fallback a rare event.

Q: The LLM makes up a policy citation that doesn't exist in the retrieved chunks. How do you catch it?

RAGAS faithfulness scoring is the primary mechanism. At eval time, faithfulness measures whether every claim in the LLM output is grounded in the retrieved chunks — a score below 0.85 triggers investigation. At inference time, the decision node's prompt explicitly instructs the LLM to cite only passages provided in the context and to use exact section references from the chunks. Post-inference, a lightweight verification step can check that every citation string in the output appears in the retrieved chunks — a simple substring match, no LLM needed. Citations that fail this check are flagged in the audit record as 'unverified_citation'.

Q: Why is the FOIR threshold 0.50 and not 0.55 as in some RBI guidelines?

0.55 is mentioned in some RBI guidance as the outer limit for salaried borrowers in certain segments. LendFlow defaults to 0.50 because: (1) Vehicle lending is an asset-backed category where conservative FOIR limits reduce default risk on a depreciating asset. (2) Many NBFCs apply 0.50 as their internal policy even when RBI guidance permits 0.55. (3) The threshold is fully configurable in config.py — a specific NBFC can set FOIR_APPROVE_THRESHOLD=0.55 without code changes. The 0.50 default represents a conservative but RBI-compliant starting point.

Q: How do you prevent the pipeline from approving a loan for a politically exposed person (PEP)?

PEP screening is not implemented in the current portfolio version but the architecture supports it cleanly. Node 2 (policy_check) is the right place to add PEP screening: after PII redaction in Node 1 stores the pii_map (name → alert_id mapping), Node 2 can call a PEP database lookup using the name from the pii_map (in a secure PII context) and add 'pep_flag' to the policy_check_result. A PEP flag would trigger HARD_REJECT or mandatory REFER with compliance escalation. The node boundary design makes this addition a localized change to policy_check only.

9. Portfolio Cross-Comparison

LendFlow is part of a portfolio of Applied LLM Systems projects. The table below compares the three main projects across key design dimensions.

Dimension	LendFlow	NexusSupply	AgentReliabilityLab
Domain	Vehicle loan underwriting	Supply chain risk intelligence	LLM agent reliability research
LLM Role	Synthesis only (Node 6)	Synthesis + FinBERT sentiment	Test subject + evaluator
Deterministic Layer	Nodes 1,2,4,5,7 — no LLM	Altman Z + XGBoost scoring	Failure taxonomy classifier
RAG / Retrieval	Hybrid: BM25+dense+reranker	FinBERT embeddings + graph	Not applicable
Evaluation Framework	RAGAS + pytest (20 cases)	MLflow + RAGAS + backtests	Custom reliability benchmarks
Regulatory Context	RBI/NBFC/NHB compliance	ESG/supply chain standards	AI safety / reliability
Key Innovation	Deterministic FOIR recomputation	Graph risk propagation	Failure mode taxonomy
Production Pattern	FastAPI + Docker + audit JSONL	FastAPI + MLflow + Streamlit	Benchmark harness + eval suite

Q: What is the common design principle across all three projects?

All three projects implement the same core principle: LLMs should synthesize and explain, not compute or decide. In LendFlow, FOIR is computed by a deterministic function — the LLM synthesizes the rationale for a decision that was already made numerically. In NexusSupply, financial health is scored by Altman Z-score and XGBoost — the LLM synthesizes a risk narrative for a supplier whose quantitative risk score was already computed. In AgentReliabilityLab, failure modes are classified by a taxonomy — the LLM is the test subject, not the evaluator. This pattern produces systems that are auditable, reliable, and trustworthy in ways that LLM-only pipelines cannot be.

Q: Which project would you productionize first and why?

LendFlow is the most production-ready: it has a defined regulatory compliance framework (RBI/NBFC), a clear customer (Indian NBFCs), measurable success metrics (FOIR accuracy, routing accuracy, PII egress), and a FastAPI + Docker deployment that can be containerized into a cloud environment with minimal changes. The addition of real bureau integration (CIBIL API) and PEP screening would close the gap to a deployable MVP. NexusSupply is production-ready in a different sense — the ML scoring pipeline is solid but the customer segment (enterprise procurement) has a longer sales cycle. AgentReliabilityLab is a research artifact, not a product.

Sidharth Kriplani · github.com/SidharthKriplani/lendflow · linkedin.com/in/sidharth-kriplani